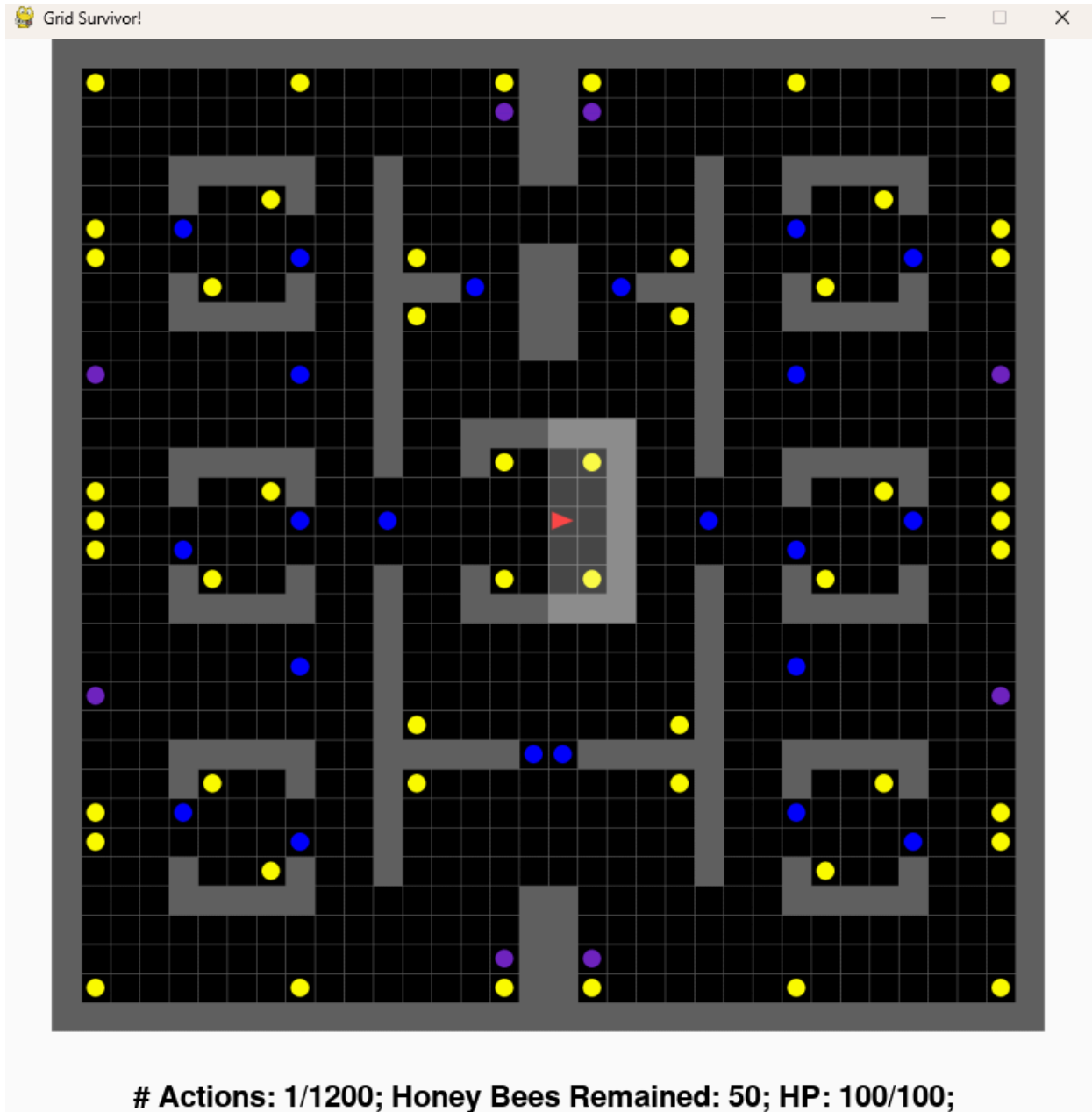


2024-02 KNU AI Team Competition Round 2

Grid Survivor!



두 번째 Team Competition은 에이전트를 노리는 말벌과 살인벌들을 피해서 꿀벌들을 구출하는 ***Grid Survivor***입니다!

Description

또 다시 플레이어는 미로에 갇혔습니다! 이 미로에는 꿀벌들도 함께 갇혀버렸습니다. 플레이어는 말벌과 살인벌들을 피해서 이 꿀벌들을 모두 구해야합니다!

Installation

*Grid Survivor*를 사용하기 위해서는 Python 3.10 이상 3.12 이하를 추천하며, 그 외의 환경에서는 정확한 동작을 보장할 수 없습니다.

*Grid Survivor*의 설치는 다음과 같이 할 수 있습니다:

```
pip install knu-rl-env==1.6
```

그 외 실행에 필요한 `gymnasium`, `pygame` 등의 라이브러리는 함께 설치되므로, 별도 설치할 필요는 없습니다.

Class and Methods

*Grid Survivor*는 다음과 같이 4개의 클래스와 함수로 구성됩니다:

knu_rl_env.grid_survivor.GridSurvivorAgent

*Grid Survivor*를 플레이하는 에이전트의 추상 클래스입니다. 아래처럼 이 추상 클래스를 상속해서 여러분만의 에이전트를 정의해보세요. 여기서 여러분이 필수로 구현해야 할 추상 함수는 상태를 받아 행동을 출력하는 **act(state)**입니다.

```
from knu_rl_env.grid_survivor import GridSurvivorAgent

class GridSurvivorRLAgent(GridSurvivorAgent):
    def act(self, state):
        '''
        Return value is one of actions following:
        - GridSurvivorAgent.ACTION_LEFT
        - GridSurvivorAgent.ACTION_RIGHT
        - GridSurvivorAgent.ACTION_FORWARD
        '''
        pass
```

knu_rl_env.grid_survivor.make_grid_survivor(show_screen)

Grid Survivor 환경을 만들어주는 함수입니다. **show_screen = True**일 경우 실제 게임 화면을 보여줍니다.

이 함수는 **gymnasium.Env**의 인스턴스를 반환하므로,

gymnasium.Env.step(action), gymnasium.Env.reset(),

gymnasium.Env.render(), gymnasium.Env.close() 의 핵심 함수 또한 활용 가능합니다.

knu_rl_env.grid_survivor.evaluate(agent)

여러분들이 훈련시킨 에이전트를 실제 시연 평가에서 보여주는 함수입니다. **agent** 인자에 여러분이 직접 구현한 에이전트를 넣으세요.

knu_rl_env.grid_survivor.run_manual()

에이전트 구현 없이 직접 플레이도 가능합니다.

- 방향키 ←: 플레이어가 좌측으로 90도 회전합니다.
- 방향키 →: 플레이어가 우측으로 90도 회전합니다.
- 방향키 ↑: 플레이어가 보는 방향으로 앞으로 1칸 이동합니다.

Environment

Observation Space

Env.step(action) 또는 Env.reset() 함수를 호출 시 **grid**와 **hit_points** 라는 필드를 가진 Dictionary가 Observation으로써 반환됩니다.

grid

미로 내의 객체들에 대한 정보로, 각 객체들은 다음과 같습니다:

- : 일반적인 바닥입니다. 플레이어는 이곳을 마음대로 움직일 수 있어요.
- : 벽입니다. 이 곳으로는 이동할 수 없고, 플레이를 이리저리 헤매게 만듭니다.
- : 플레이어입니다. 화살표의 방향이 현재 향하고 있는 방향입니다.

- : 꿀벌입니다. 플레이어가 꿀벌에 닿으면 구할 수 있습니다.
- : 말벌입니다. 플레이어가 말벌에 닿으면 체력을 20 소진합니다. 하지만, 다행히도 한번 침을 쏜 말벌은 죽고 맙니다.
- : 끔찍한 독성을 가진 살인벌입니다. 이 녀석에게 닿는 순간 곧바로 플레이어는 죽게 됩니다.

에이전트는 이러한 미로의 구성 요소들을 Numpy array 배열로 알 수 있으며, 배열의 원소는 객체 및 객체의 상태를 나타내는 알파벳 문자로 구성됩니다:

- **E**: 일반적인 바닥
- **W**: 벽
- **AL**: 좌측 방향을 보고 있는 플레이어
- **AR**: 우측 방향을 보고 있는 플레이어
- **AU**: 위 방향을 보고 있는 플레이어
- **AD**: 아랫 방향을 보고 있는 플레이어
- **B**: 꿀벌
- **H**: 말벌
- **K**: 살인벌

hit_points

플레이어의 남은 체력에 대한 정보를 int 형으로 가지고 있습니다.

Action Space

플레이어는 다음과 같은 3개의 행동을 할 수 있으며, 에이전트의

구현체 `knu_rl_env.GridSurvivorAgent`에 각 행동을 상수로 지정해두었습니다:

- `knu_rl_env.GridSurvivorAgent.ACTION_LEFT`: 플레이어가 좌측으로 90도 회전합니다.
- `knu_rl_env.GridSurvivorAgent.ACTION_RIGHT`: 플레이어가 우측으로 90도 회전합니다.
- `knu_rl_env.GridSurvivorAgent.ACTION_FORWARD`: 플레이어가 보는 방향으로 앞으로 1칸 이동합니다.

Reward Space

어떤 행동이든 보상의 양을 0으로 설정했습니다. 여러분들이 자유롭게 보상을 설계해보세요!

Episode End

꿀벌을 모두 구하거나, 체력이 0이 되거나, 말벌에 닿거나, 1,200회 이상 행동을 하게 되면 에피소드가 종료됩니다.

Submission

제출해야 하는 파일은 두 개입니다.

Implementation

[구현 템플릿](#)에 에이전트의 추상 클래스 `knu_rl_env.GridSurvivorAgent` 를 상속하여 자신만의 에이전트를 정의하고, 이 에이전트를 훈련하는 코드를 구현하세요.

```
from knu_rl_env.grid_suvivor import GridSurvivorAgent, make_grid_suvivor, evaluate

'''
knu_rl_env.grid_adventure.GridAdventureAgent을 상속해서
자신만의 에이전트를 구현하세요.
'''

class GridSurvivorRLAgent(GridSurvivorAgent):
    def act(self, state):
        '''
        다음 중 하나를 반환해야 합니다:
        - GridSurvivorAgent.ACTION_LEFT
        - GridSurvivorAgent.ACTION_RIGHT
        - GridSurvivorAgent.ACTION_FORWARD
        '''
        pass

'''
에이전트를 훈련하는 코드를 구현하세요.
'''

def train():
    '''
    Grid Survivor 환경은 다음과 같이 생성할 수 있습니다.
    '''
    env = make_grid_suvivor(
        show_screen=True # or, False
    )
    '''
    여기서부터는 이 환경에 대해서 에이전트를 훈련시키는 코드가 필요합니다.
    '''
```

Dependent Libraries

에이전트 구현 시에 사용한 의존성 라이브러리의 목록을 다음 명령어로 생성된

`requirements.txt` 파일로 제출해야 합니다.

```
pip list -format=freeze > requirements.txt
```

Evaluation

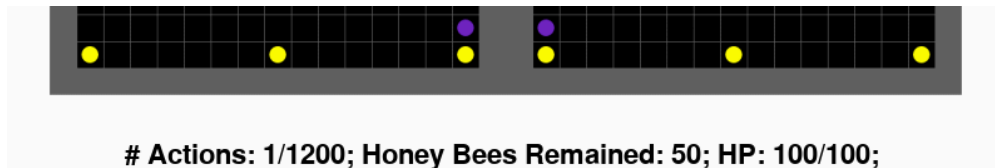
Team Competition은 다음의 코드를 실행하여 플레이한 결과로 결정됩니다

(별도의 실행 코드는 인정하지 않습니다):

```
from knu_rl_env.grid_survivor import evaluate

if __name__ == '__main__':
    agent = '''여러분이 정의하고 학습시킨 에이전트를 불러오는 코드를 넣으세요.'''
    evaluate(agent)
```

Grading Criteria



최소한의 행동을 수행하면서 모든 꿀벌을 구하는 것이 목표이며, 세부 평가 기준은

다음과 같습니다:

- 남은 꿀벌의 수 (Honey Bees Remained)가 적을수록 좋음
- 남은 꿀벌의 수가 같을 시, 행동 수행 횟수 (# Actions)가 작을수록 좋음
- 남은 꿀벌의 수와 행동 수행 횟수가 같을 시, 남은 체력 (HP)이 많을 수록
좋음

총 3번의 시도를 할 수 있으며, 세 시도의 결과를 합산하여 위 기준에 따라 순위를 정합니다. 순위에 따른 점수는 다음과 같습니다:

- 1위: 30%
- 2위: 27%
- 3위: 24%
- 4위: 21%
- 5위: 18%
- 6위: 15%
- 7위: 12%
- 8위: 9%

0 Points Policy

단, 다음 중 하나에 해당하는 경우에는 무조건 0% 처리가 됩니다.

- 다른 버전의 *Grid Survivor* 환경을 활용하는 경우 (1.6 버전으로 설치 및 과제 수행할 것)
- 제출 마감 내에 제출물 (에이전트 구현체 및 **requirements.txt**)을 이투리에 제출하지 않은 경우 (지연 제출 또한 허용 안됨)

- requirements.txt에서 정의된 모든 의존성 라이브러리를 설치했음에도 불구하고 제출한 코드의 실행이 되지 않는 경우
- 제출한 코드의 실행 결과로 학습된 에이전트가 실제 시연 결과와 크게 다른 경우
- 시연을 하지 않거나, 시연이 불가능한 경우
 - 자신의 팀이 시연을 하는 일자에 출석하지 않는 경우, 해당 학생만 0% 처리
- 정책 또는 가치 함수를 학습하지 않고, 직접 손으로 지정한 정책 또는 가치 함수를 사용하는 경우
- 환경을 바꾸는 경우
- 자신이 구현하지 않은 별도의 강화 학습 구현체 또는 강화 학습 라이브러리를 활용하는 경우

Slot Assignments, Submission Deadline, and Results

Early-Bird Slot							Regular-Bird Slot						
● 제출 마감: 2024년 11월 25일 08:59 ● 시연 시간: 2024년 11월 25일 09:00 -							● 제출 마감: 2024년 11월 28일 08:59 ● 시연 시간: 2024년 11월 28일 09:00 -						
순서	팀 이름	시도	남은 꿀벌 수	행동 수	체력	순위	순서	팀 이름	시도	남은 꿀벌 수	행동 수	체력	순위
1	K&L	1회차					1	우직하게	1회차				

		2회차							2회차				
		3회차							3회차				
		합산							합산				
2	이호준	1회차					2	파워레인저 정글포스	1회차				
		2회차							2회차				
		3회차							3회차				
		합산							합산				
3	함성준	1회차					3	A	1회차				
		2회차							2회차				
		3회차							3회차				
		합산							합산				
4	나또진	1회차					4	파워레인저 매직포스	1회차				
		2회차							2회차				
		3회차							3회차				
		합산							합산				
5	김다원	1회차					5	초코비	1회차				
		2회차							2회차				
		3회차							3회차				
		합산							합산				
6	임재은	1회차					6	미배정	1회차				
		2회차							2회차				
		3회차							3회차				
		합산							합산				

7	제주	1회차					7	Team3	1회차				
		2회차							2회차				
		3회차							3회차				
		합산							합산				
8	ICN	1회차					8	학교앞 고속도로	1회차				
		2회차							2회차				
		3회차							3회차				
		합산							합산				